4.6 Step by Step

Now you should know where you are in the running trace and be able to print the available variables. But lets continue and move on with the application execution.

Use `step` (abbreviated `s`) to continue running your program until the next logical stopping point and return control to the debugger.
You may also use `next` which is similar to step, but function or method calls that appear within the line of code are executed without stopping.
You can also use `step n` or `next n` to move forwards `n` steps at once.

The difference between `next` and `step` is that `step` stops at the next line of code executed, doing just a single step, while `next` moves to the next line without descending inside methods.
For example, consider the following situation:

```
Started GET "/" for 127.0.0.1 at 2014-04-11 13:39:23 +0200
Processing by ArticlesController#index as HTML

[1, 8] in /home/davidr/Proyectos/test_app/app/models/article.rb
   1: class Article < ActiveRecord::Base
   2:
   3:   def self.find_recent(limit = 10)
   4:     byebug
=> 5:     where('created_at > ?', 1.week.ago).limit(limit)
   6:   end
   7:
   8: end

(byebug)
```

If we use `next`, we want go deep inside method calls. Instead, byebug will go to the next line within the same context. In this case, this is the last line of the method, so `byebug` will jump to next next line of the previous frame.

```
(byebug) next
Next went up a frame because previous frame finished

[4, 13] in
/PathTo/project/test_app/app/controllers/articles_controller.rb
    4:   # GET /articles
    5:   # GET /articles.json
    6:   def index
    7:     @articles = Article.find_recent
    8:
=>  9:     respond_to do |format|
   10:       format.html # index.html.erb
   11:       format.json { render json: @articles }
   12:     end
   13:   end

(byebug)
```

If we use `step` in the same situation, we will literally go the next ruby instruction to be executed. In this case, the activesupport's `week` method.

```
(byebug) step

[50, 59] in /PathToGems/activesupport-
5.0.0/lib/active_support/core_ext/numeric/time.rb
   50:     ActiveSupport::Duration.new(self * 24.hours, [[:days,
self]])
   51:   end
   52:   alias :day :days
```